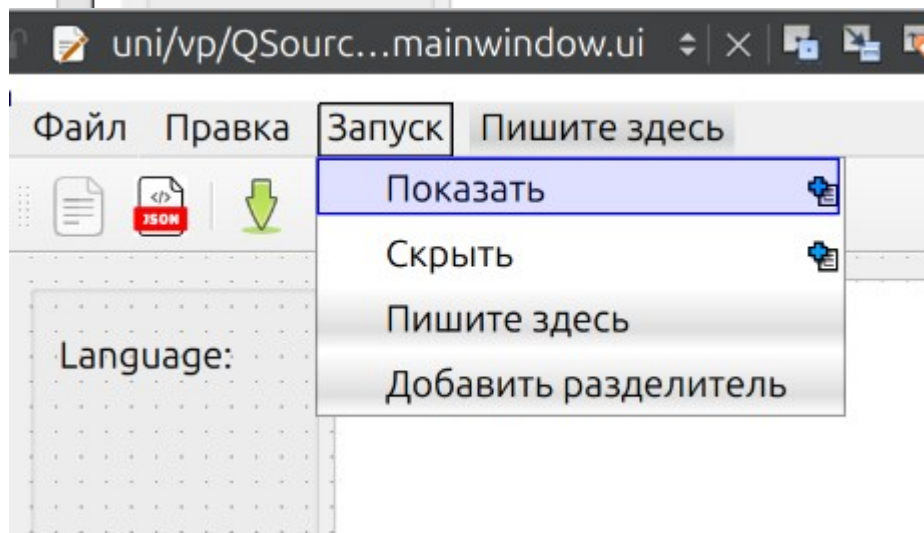
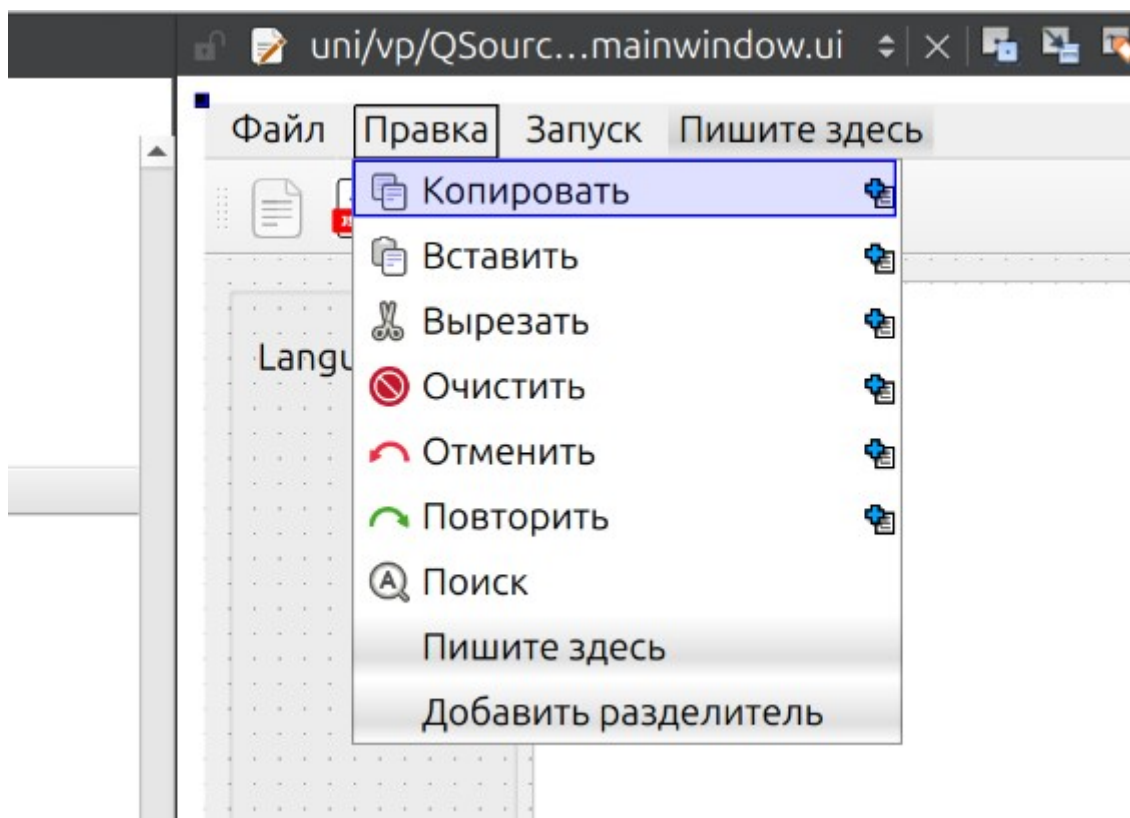
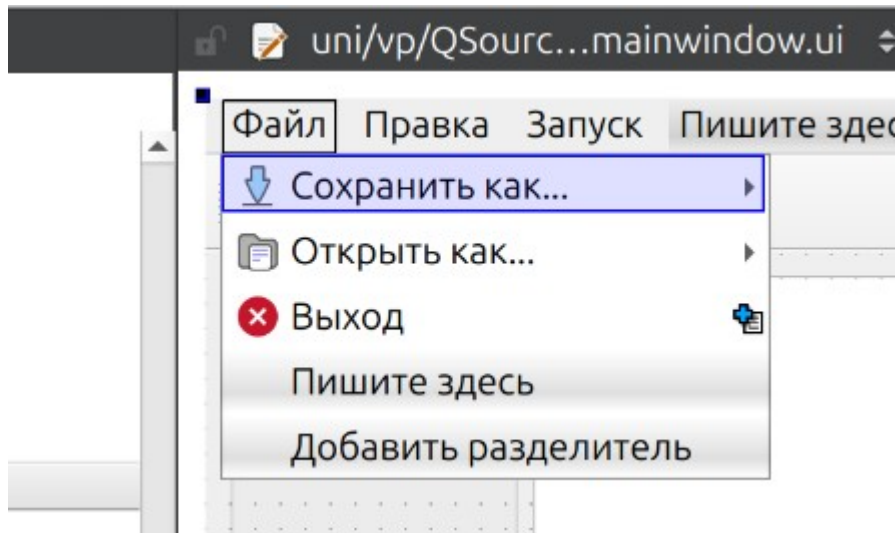


Практическая работа №2

Выполнила: Ушакова Е.А. гр. 3П-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке
<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Добавлены пункты меню по заданию:



2) Действия из меню Правка связаны со слотами виджета PlainTextEdit через интерфейс Дизайнера

Отправитель	Сигнал	Получатель	Слот
undo	triggered()	plainTextEdit	undo()
redo	triggered()	plainTextEdit	redo()
paste	triggered()	plainTextEdit	paste()
cut	triggered()	plainTextEdit	cut()
copy	triggered()	plainTextEdit	copy()
clear	triggered()	plainTextEdit	clear()

3) Реализованы функции для сохранения и открытия текстового документа

```

306 void MainWindow::on_SaveTXT_triggered () {
307     QString fileName = QFileDialog::getSaveFileName (this, "Сохранить TXT файл", "", "TXT файлы *.txt");
308     if (fileName.isEmpty()) {
309         return;
310     }
311     QFile file(fileName);
312     if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
313         ui->statusbar->showMessage("Ошибка: не удалось открыть файл для записи.", 3000);
314         return;
315     }
316     QString text = ui->plainTextEdit->toPlainText();
317     QTextStream out(&file);
318
319     out << text;
320
321     file.close();
322
323     isUnnamed = false;
324     currentFile = fileName;
325     ui->statusbar->showMessage("Файл " + fileName + " сохранен", 3000);
326 }
327
328 void MainWindow::on_OpenTXT_triggered () {
329     QString fileName = QFileDialog::getOpenFileName (this, "Сохранить TXT файл", "", "TXT файлы *.txt");
330     if (fileName.isEmpty()) {
331         return;
332     }
333     QFile file(fileName);
334     if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
335         ui->statusbar->showMessage("Ошибка: не удалось открыть файл для записи.", 3000);
336         return;
337     }
338     ui->plainTextEdit->setPlainText(file.readAll());
339
340     file.close();
341
342     isUnnamed = false;
343     currentFile = fileName;
344     ui->statusbar->showMessage("Файл " + fileName + " открыт", 3000);
345 }

```

оба метода вызывают стандартный диалог, из которого получают имя файла для открытия или сохранения, после чего считывают/записывают данные и запоминают имя файла в переменную currentFile.

4) Сохранение в JSON и считывание из него происходят схожим образом, но при сохранении

```
348 void MainWindow::on_SaveJSON_triggered () {
349     QString fileName = QFileDialog::getSaveFileName (this, "Сохранить JSON файл", "", "JSON файлы *.json");
350     if (fileName.isEmpty()) {
351         return;
352     }
353     QFile file(fileName);
354     if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
355         ui->statusbar->showMessage("Ошибка: не удалось открыть файл для записи.", 3000);
356         return;
357     }
358     QJsonObject root;
359     root["language"] = ui->langComboBox->currentText();
360     root["theme"] = ui->themeComboBox->currentData().toInt();
361     root["text"] = ui->plainTextEdit->toPlainText();
362
363     QJsonDocument doc(root);
364     file.write(doc.toJson());
365
366     file.close();
367
368     isUnnamed = false;
369     currentFile = fileName;
370     ui->statusbar->showMessage("Файл " + fileName + " сохранен", 3000);
371 }
372 }
```

к тексту добавляются параметры Language и theme, которые при открытии файла передают значения темы и языка файла в соответствующие поля программы.

```
373 void MainWindow::on_OpenJSON_triggered () {
374     QString fileName = QFileDialog::getOpenFileName (this, "Сохранить JSON файл", "", "JSON файлы *.json");
375     if (fileName.isEmpty()) {
376         return;
377     }
378     QFile file(fileName);
379     if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
380         ui->statusbar->showMessage("Ошибка: не удалось открыть файл.", 3000);
381         return;
382     }
383     QJsonParseError error;
384     QJsonDocument doc = QJsonDocument::fromJson(file.readAll(), &error);
385     file.close();
386     if (error.error != QJsonParseError::NoError) {
387         ui->statusbar->showMessage("Ошибка парсинга.", 3000);
388         return;
389     }
390
391     QJsonObject obj = doc.object();
392     QString lang = obj["language"].toString();
393     int theme = obj["theme"].toInt();
394     QString text = obj["text"].toString();
395
396     ui->plainTextEdit->setPlainText(text);
397
398     if (!lang.isEmpty()) {
399         ui->langComboBox->setCurrentText(lang);
400     }
401
402     ui->themeComboBox->setCurrentIndex(theme);
403
404     isUnnamed = false;
405     currentFile = fileName;
406     ui->statusbar->showMessage("Файл " + fileName + " сохранен", 3000);
407 }
408 }
```

5) При выходе из приложения запускается проверка на наличие несохраненных изменений и в случае их наличия создается окно сообщения с предложением сохранить изменения.

```

134 }
135
136 void MainWindow::closeEvent(QCloseEvent *event) {
137     if (!ui->plainTextEdit->document()->isModified()) {
138         event->accept();
139         return;
140     }
141     QMessageBox exMsg;
142     exMsg.setText("Остались несохраненные изменения. Вы уверены что хотите выйти?");
143     QPushButton * saveTXT = exMsg.addButton("save as TXT", QMessageBox::AcceptRole);
144     QPushButton * saveJSON = exMsg.addButton("save as JSON", QMessageBox::AcceptRole);
145     exMsg.addButton(QMessageBox::Discard);
146     exMsg.addButton(QMessageBox::Cancel);
147     exMsg.setDefaultButton(QMessageBox::Cancel);
148
149     int ret = exMsg.exec();
150
151     switch (ret) {
152     case QMessageBox::Discard:
153         ui->statusbar->showMessage("Disgard");
154         event->accept();
155         break;
156     case QMessageBox::Cancel:
157         event->ignore();
158         ui->statusbar->showMessage("Cancel", 3000);
159         break;
160     default:
161         if (exMsg.clickedButton() == saveTXT) {
162             on_SaveTXT_triggered();
163         } else if (exMsg.clickedButton() == saveJSON) {
164             on_SaveJSON_triggered();
165         }
166         event->accept();
167         break;
168     }
169 }

```

6) Для поиска и замены создан отдельный кастомный диалог со следующими приватными слотами:

```

40
41 void SearchDialog::on_findButton_clicked () {
42     emit findNext(findLineEdit->text());
43 }
44
45 void SearchDialog::on_replaceButton_clicked () {
46     emit replaceText(findLineEdit->text(), replaceLineEdit->text());
47 }
48
49 void SearchDialog::on_replaceAllButton_clicked () {
50     emit replaceAll(findLineEdit->text(), replaceLineEdit->text());
51 }
52

```

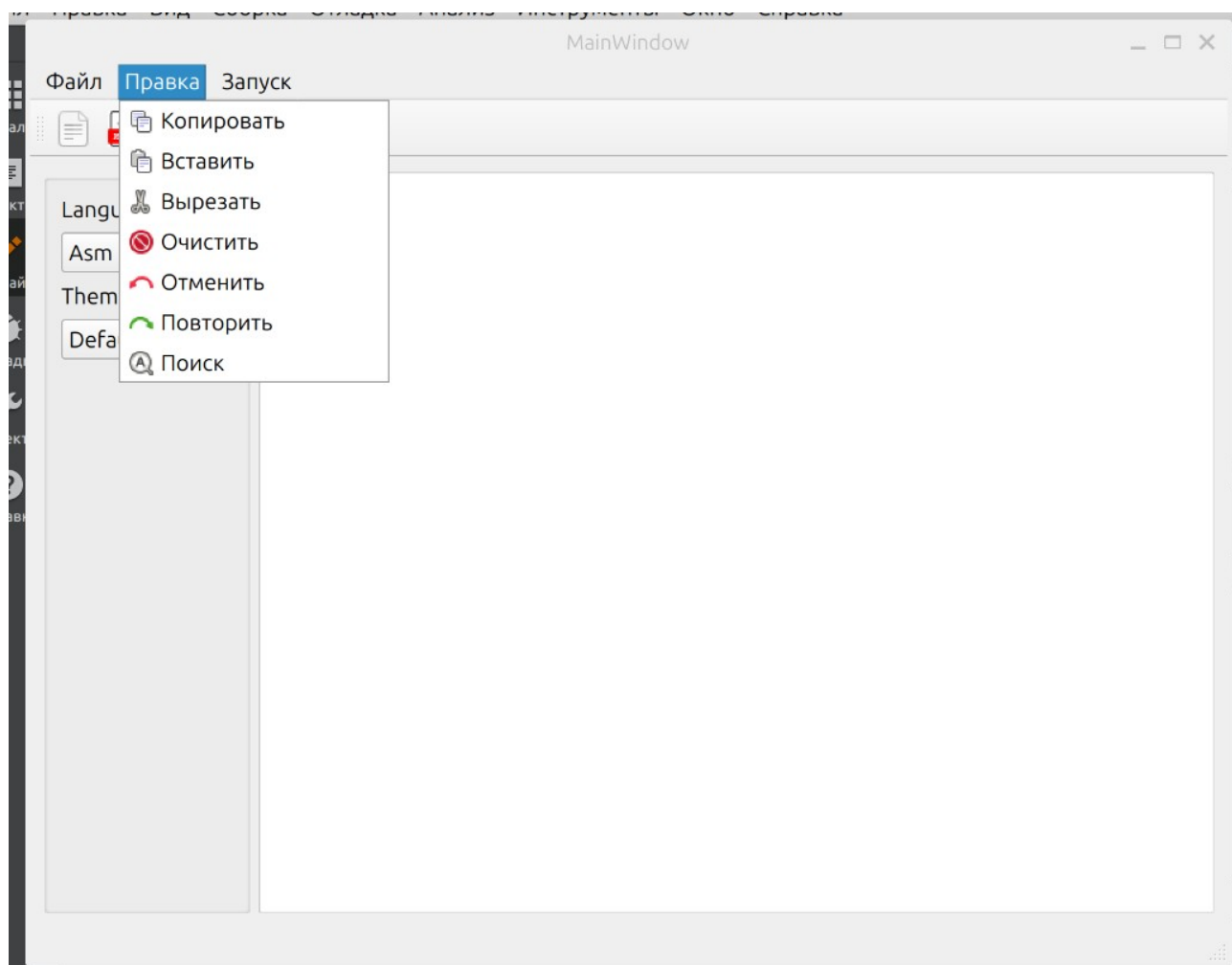
7) В основном окне к этим слотам подключены функции:

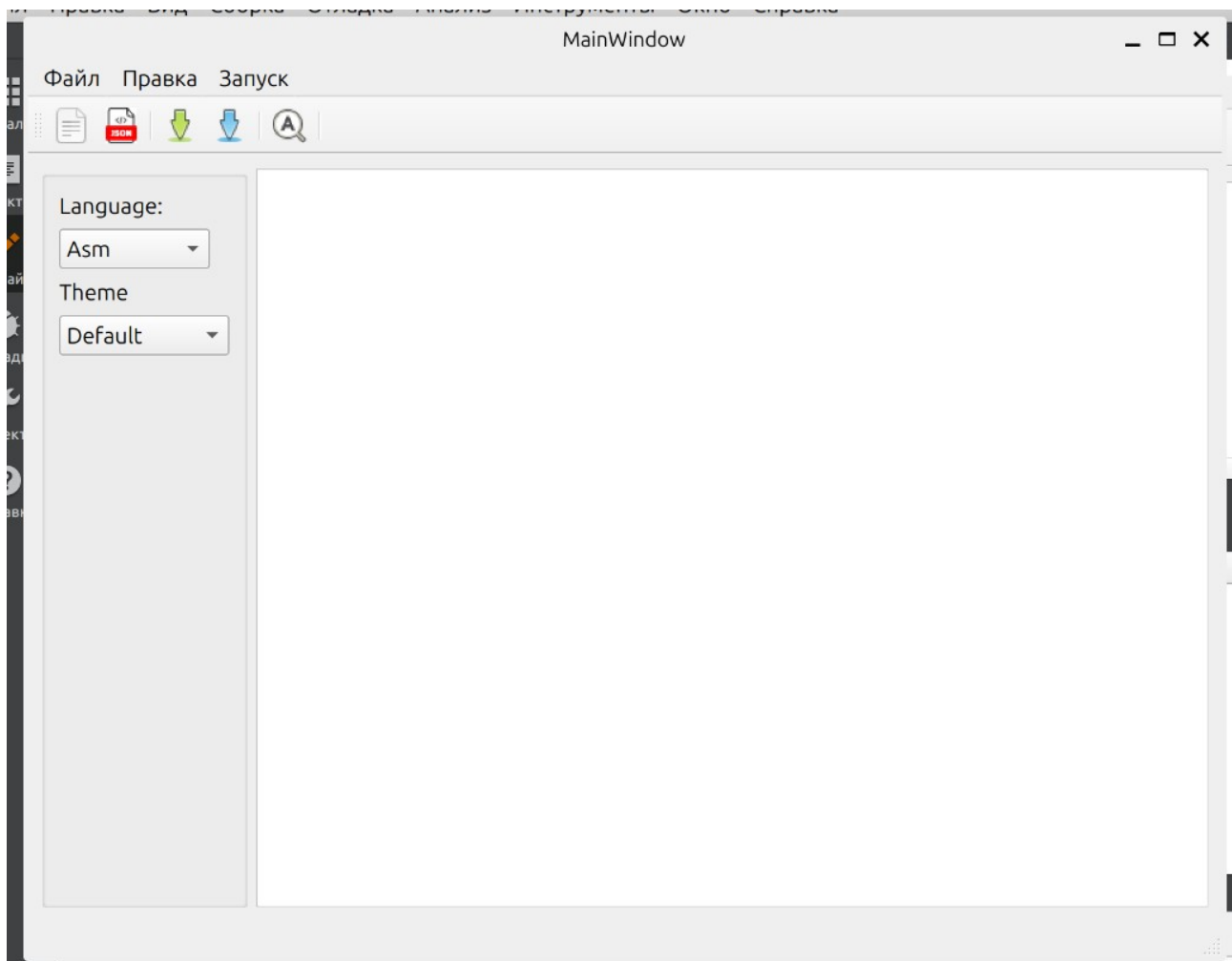

```

243 void MainWindow::onSearch() {
244     SearchDialog dialog(this);
245     connect(&dialog, &SearchDialog::findNext, this, &MainWindow::onFindNext);
246     connect(&dialog, &SearchDialog::replaceText, this, &MainWindow::onReplaceText);
247     connect(&dialog, &SearchDialog::replaceAll, this, &MainWindow::onReplaceAll);
248     dialog.exec();
249 }
250
251 void MainWindow::onFindNext(const QString &text) {
252     QTextCursor cursor = ui->plainTextEdit->textCursor();
253     QString content = ui->plainTextEdit->toPlainText();
254     int pos = content.indexOf(text, cursor.position(), Qt::CaseSensitive);
255
256     if (pos >= 0) {
257         cursor.setPosition(pos);
258         cursor.movePosition(QTextCursor::Right, QTextCursor::KeepAnchor, text.length());
259         ui->plainTextEdit->setTextCursor(cursor);
260         ui->statusbar->showMessage("Найдено: " + text, 3000);
261     } else {
262         pos = content.indexOf(text, 0, Qt::CaseSensitive);
263         if (pos >= 0) {
264             cursor.setPosition(pos);
265             cursor.movePosition(QTextCursor::Right, QTextCursor::KeepAnchor, text.length());
266             ui->plainTextEdit->setTextCursor(cursor);
267             ui->statusbar->showMessage("Поиск с начала. Найдено: " + text, 3000);
268         } else {
269             QMessageBox::information(this, "Поиск", "Совпадений не найдено");
270             ui->statusbar->showMessage("Совпадений не найдено", 3000);
271         }
272     }
273 }
274
275 }
276
277 void MainWindow::onReplaceText(const QString &find, const QString &replace) {
278     QTextCursor found = ui->plainTextEdit->textCursor();
279     QString selected = found.selectedText();
280
281     if (!selected.isEmpty() && selected == find) {
282         found.insertText(replace);
283         ui->plainTextEdit->setTextCursor(found);
284     } else {
285         onFindNext(find);
286         found = ui->plainTextEdit->textCursor();
287         selected = found.selectedText();
288         if (!selected.isEmpty() && selected == find) {
289             found.insertText(replace);
290             ui->plainTextEdit->setTextCursor(found);
291         }
292     }
293
294     highlighter->rehighlight();
295 }
296
297 void MainWindow::onReplaceAll(const QString &find, const QString &replace) {
298     QString content = ui->plainTextEdit->toPlainText();
299     QString newContent = content.replace(find, replace, Qt::CaseSensitive);
300     ui->plainTextEdit->setPlainText(newContent);
301     highlighter->rehighlight();
302     QString message = (QString("Все вхождения '%1' заменены на '%2'").arg(find).arg(replace));
303     ui->statusbar->showMessage(message);
304 }

```

Дополнительное задание: добавлены иконки пунктам меню и toolbar





Ответы на контрольные вопросы

Как работает механизм подсветки синтаксиса в QSyntaxHighlighter?

Создаётся наследник QSyntaxHighlighter, переопределяется метод highlightBlock(const QString &text). Внутри анализируется переданный блок текста, с помощью setFormat(start, length, QTextCharFormat) задаются цвета/стили для лексем (ключевые слова, строки, комментарии и т.д.).

Что делает метод rehighlight() и почему он важен при изменении языка или текста?

rehighlight() перезапускает подсветку для всего документа, заново вызывая highlightBlock() для каждого блока. Он важен, потому что при смене языка программирования или полной загрузке нового текста старая подсветка становится неактуальной, и её нужно полностью пересчитать.

Какие сигналы и слоты используются в диалоге поиска и замены?

Основные:

findNext() поиск следующего вхождения.

replaceText() — замена текущего найденного.

replaceAll() — замена всех вхождений.

Сигналы от кнопок (clicked()) подключаются к этим слотам.

Что хранится в JSON-файле при сохранении документа?

содержимое текста ("content"), выбранный язык программирования ("language").

Как восстанавливается язык программирования при открытии JSON-файла?

При загрузке JSON парсится поле "language" (например, "Python"), по строке находится соответствующий индекс в QComboBox, устанавливается setCurrentIndex(), после чего вызывается rehighlight() для применения правильной подсветки синтаксиса.

Как работает связь между ComboBox и подсветкой кода?

При выборе нового языка в QComboBox генерируется сигнал currentTextChanged. В слоте этот сигнал вызывает метод, который меняет язык в объекте подсветки и затем вызывает rehighlight() для документа.

Какие классы Qt используются для создания пользовательского диалога поиска и замены?

QDialog — базовый класс для окна; QLineEdit — поля ввода для поиска и замены; QPushButton — кнопки "Найти", "Заменить", "Заменить всё", "Отмена"

Как применяется QTextCursor в данном проекте?

Для навигации и редактирования текста: поиск вхождений (document()->find()), выделение найденного, удаление/вставка текста при замене, получение/установка позиции курсора, применение форматирования.

Как проверить, были ли сделаны изменения в документе перед выходом?

Использовать QTextDocument::modified() или флаг isModified в главном окне. При закрытии окна (событие closeEvent()) проверяется этот флаг. Если true — показывается QMessageBox с вопросом "Сохранить изменения?".

Какие классы и методы Qt используются для работы с JSON?

QJsonDocument — парсинг/генерация JSON.

QJsonObject — работа с объектами.

QJsonArray — работа с массивами.

QJsonValue — универсальное значение.

Методы: QJsonDocument::fromJson(), QJsonDocument::toJson(), QJsonObject::insert(), value(), contains().